

A Python based approach to animate geometry and data

Shubham Tiwari

Supervisor: Dr. J. V. Ramani

Co-Supervisor: Ms. Ishver Rani Manchanda

Dyal Singh College, University of Delhi

April 2026

Project Abstract

- **Framework:** Python-based computational environment for converting static equations into interactive animations.
- **Stack:** Leverages NumPy for vectorized array processing and Matplotlib as the rendering engine.
- **Scope:** Covers 2D kinematics, L_p norm morphing, and complex 3D differential topology (Minimal surfaces like Boy's Surface).
- **Goal:** Enhancing student understanding of abstract spatial transformations through visualization.

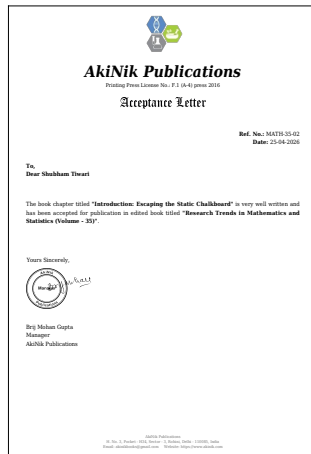
Project Motivation: The Visualization Problem

- **The Gap:** Transitioning from basic mathematics to more abstract concepts (Trigonometry/Calculus/Algebra) often leads to a loss of interest due to lack of intuition.
- **The Solution:** Animation fills the gap between theory and understanding by showing *how* shapes evolve.

"The most important thing a mathematician can do is to show that mathematics is a human activity... one of the most creative fields." — William Thurston

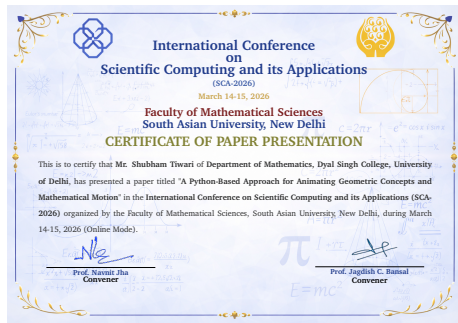
Acceptance Letter

- **Title:** "Introduction: Escaping the Static Chalkboard"
- **Publisher:** AkiNik Publications
- **Book:** Research Trends in Mathematics and Statistics (Vol. 35)
- **Date:** April 25, 2026



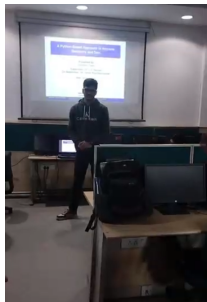
SCA-2026 Presentation

- **Conference:** Int. Conf. on Scientific Computing and its Applications.
- **Host:** South Asian University, New Delhi.
- **Paper:** "A Python-Based Approach for Animating Geometric Concepts..."



Research Progress: Monthly Reviews

- **Jan:** Initial 3D logic.
- **Feb:** 3D Surface Rendering.
- **Mar:** Topological Morphing.
- **Apr:** Final Review Conclusion.



$$P(t) = (x(t), y(t), z(t))$$

- Time introduces motion
- Geometry defines structure
- Transformation controls behavior

Animation = Geometry + Transformation + Time [5]

Parametrization

$$x = f(t), \quad y = g(t)$$

- Converts static equations into motion [3]
- Fundamental tool for curves

[Show curve tracing animation here] [6]

Sliding Ladder Problem

$$x^2 + y^2 = L^2$$

$$x = L \cos \theta, \quad y = L \sin \theta$$

Midpoint:

$$\left(\frac{L}{2} \cos \theta, \frac{L}{2} \sin \theta \right)$$

$$x^2 + y^2 = \left(\frac{L}{2} \right)^2 \quad [3]$$

Midpoint traces a circle

[Show ladder animation here]

$$P' = RP + T$$

- R : rotation matrix
- T : translation vector

[Show rotating cube animation here] [1]

Rotation Matrix

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Preserves distance
- Preserves angles
- Defines rigid motion [1]

Linear algebra controls space

Distance Preservation of Rotation Matrices

Let $P' = RP$ be the rotated vector. The squared norm of the rotated vector is:

$$\|P'\|^2 = (RP)^T(RP)$$

Expanding the transpose:

$$\|P'\|^2 = P^T R^T RP$$

Since the rotation matrix is orthogonal, $R^T R = I$, we obtain:

$$\|P'\|^2 = P^T I P = P^T P$$

Thus:

$$\|P'\| = \|P\| \quad [1]$$

- Rotation preserves magnitude
- Distance from origin remains constant
- Shape does not deform

The Power of Orthogonality in Animation

The orthogonal property $R^T R = I$ (meaning $R^{-1} = R^T$) is computationally profound:

- **Computational Efficiency:** Finding an inverse matrix is usually expensive. For orthogonal rotation matrices, reversing a rotation is just a simple matrix transpose, making frame-by-frame rendering incredibly fast [7].
- **Rigid Body Integrity:** Because length and angles are preserved, 3D models do not stretch, shear, or distort during complex tumbling [1].
- **Numerical Stability:** Repeated matrix multiplications can accumulate floating-point errors. Orthogonal matrices naturally resist degradation over time in continuous simulations.

The Animation Pipeline I: Theory to Data

Step 1: Mathematical Formulation

Converting implicit geometry $F(x, y, z) = 0$ into time-dependent parametric vector functions $P(t) = [x(t), y(t), z(t)]^T$.



Step 2: Discretization

Slicing continuous domains into finite NumPy arrays.

```
theta = np.linspace(0, 2*np.pi, frames)
```

- **Vectorization:** Applying functions to entire arrays at once for performance.
- **Discrete Mapping:** Finding the optimal Δt to balance smoothness vs. speed.

The Animation Pipeline II: Data to Motion

Step 3: Transformation

Applying $SO(3)$ rotation matrices (R) and translation vectors (T).

$$P' = RP + T$$



Step 4: Rendering

The `update(frame)` function injects new coordinates into the Matplotlib canvas sequentially.

- **Rigid Motion:** Matrices preserve L_2 norm (shape integrity).
- **Frame-by-Frame:** Data is redrawn inside the `FuncAnimation` loop.

Parametric Surfaces

$$x = f(u, v), \quad y = g(u, v), \quad z = h(u, v)$$

- General representation of surfaces [2]
- Extends curves to higher dimensions

[Show surface animation here] [6]

$$H = 0$$

- Zero mean curvature surfaces [2]
- Arise in optimization and physics
- Example: soap films

Importance:

- Differential Geometry
- Material science
- Computer graphics

Examples: Minimal Surfaces

Catenoid

$$x = a \cosh(v) \cos(u), \quad y = a \cosh(v) \sin(u), \quad z = av$$

Helicoid

$$x = u \cos v, \quad y = u \sin v, \quad z = cv$$

- Different parametrizations
- Same geometric principle [2]

[Show catenoid/helicoid animation here]

Articulated Motion: Human Movement

- Human motion can be modeled as a system of interconnected points (joints) and line segments (limbs).
- Each joint is represented as a point in $\mathbb{R}^3$, and motion is generated using transformations over time:

$$P_i(t) = R_i(t)P_i + T_i(t)$$

- Complex motion emerges from simple rotational transformations applied hierarchically. [1]
- This forms the basis of **kinematics**, widely used in computer graphics, robotics, and animation.

[Show articulated human motion animation] [7]

- Parametrization $\rightarrow$ motion
- Linear algebra $\rightarrow$ transformations
- Geometry $\rightarrow$ structure
- Differential geometry $\rightarrow$ surfaces

Complex motion arises from simple equations [4]

$$\text{Ray}(t) = P + td$$

- Rays simulate vision
- Distance determines height
- Height $\propto \frac{1}{\text{distance}}$ [3]

3D illusion from purely 2D mathematics

Core Development Pipeline

- Developed the core 4-step pipeline: Formulation → Discretization → Transformation → Rendering.
- Built dynamic 2D engines using vectorized NumPy arrays.
- Optimized memory using `blit=True` and data injection.

```
# 2. DISCRETIZE THE DOMAIN
u = np.linspace(-10, 10, 100)
t = np.linspace(-10, 10, 100)

# 4. UPDATE FUNCTION (CORE ENGINE)
def update(frame):
    k = frame
    # Evaluate parametric coordinates dynamically
    y_evaluated = k * np.sin(u)

    # Inject newly calculated coordinates
    straight_line.set_data(u, y_evaluated)
    return straight_line,
```

Advanced Spatial Computations

- Programmed $SO(3)$ rotation matrices (R_x, R_y, R_z) from scratch.
- Applied continuous matrix multiplication over bivariate parametric meshes (e.g., Ellipsoids, Tori).
- Prevented RAM memory leaks by clearing 3D axes (`ax.clear()`) frame-by-frame.

3. LINEAR ALGEBRA ROTATION OPERATOR

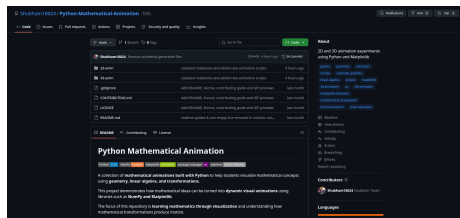
```
def rotate_x(x, y, z, angle):  
    cos_a, sin_a = np.cos(angle), np.sin(angle)  
    x_new = x  
    y_new = cos_a * y - sin_a * z  
    z_new = sin_a * y + cos_a * z  
    return x_new, y_new, z_new
```

4. UPDATE FUNCTION (3D ENGINE)

```
def update(frame):  
    ax.clear() # Prevent memory leaks  
    ax.set_box_aspect([1,1,1]) # Enforce proportions  
  
    angle = frame * 0.05  
    Xr, Yr, Zr = rotate_x(X, Y, Z, angle)  
    ax.plot_wireframe(Xr, Yr, Zr, color="magenta")  
)
```

Project Hosting: GitHub Repository

- **Centralized Hosting:** All source code, Jupyter Notebooks, and rendered assets are publicly available.
- **Version Control:** Organized history of development from initial 2D plots to complex 3D manifolds.
- **Accessibility:** Open-source license for academic collaboration.



Robust Environment Control: uv

- **Zero Dependency Issues:** Entire project stack managed via the Rust-based uv manager.
- **Reproducibility:** Isolated environments ensure the code runs flawlessly across Linux, Windows, and Mac.
- **Speed:** Dependency resolution and installation 10-100x faster than standard pip.

```
uv sync → uv run jupyter notebook
```

Overcoming Technical Difficulties

- **CPU-Bound Rendering Limits:** Matplotlib renders on the CPU, causing FPS drops on dense topologies (like the Helicoid). *Solution:* Carefully optimized `np.linspace` resolution.
- **Z-Buffering (Depth Sorting) Artifacts:** Matplotlib uses a 2D "painter's algorithm" to fake 3D, leading to background polygons bleeding through. *Solution:* Manually enforced viewing angles.
- **Memory Leaks:** Continuous redrawing crashed the RAM. *Solution:* Implemented strict `ax.clear()` garbage collection per frame.
- **Aspect Ratio Distortion:** Auto-scaling warped 3D rotating spheres into ellipsoids. *Solution:* Hardcoded `ax.set_box_aspect([1,1,1])` to force isometric dimensions.

- **GPU Migration:** Shifting from CPU-bound Matplotlib to Manim or Three.js for 60 FPS real-time rendering.
- **GUI Development:** Creating a PyQt-based application for students to input their own parametric equations via sliders.
- **Physics Engines:** Integrating SciPy ODE solvers for mass, gravity, and collision detection.

Conclusion

- Mathematics can generate motion
- Linear algebra defines space
- Parametrization creates geometry
- Visualization enhances understanding

Mathematics is not static — it is alive.

References I



Friedberg, S. H., Insel, A. J., & Spence, L. E. (2019). *Linear Algebra* (5th ed.). Pearson Education.



Kobayashi, S. (2019). *Differential Geometry of Curves and Surfaces*. Springer.



Loney, S. L. (1895). *The Elements of Coordinate Geometry*. Macmillan.



Kreyszig, E. (1989). *Introductory Functional Analysis with Applications*. Wiley.



Python Software Foundation (2024). *Python Language Reference*.
<https://docs.python.org/>



Matplotlib Developers (2024). *Matplotlib Documentation*.
<https://matplotlib.org/>



NumPy Developers (2024). *NumPy Documentation*.
<https://numpy.org/>

"Nature's patterns are the most beautiful, but they are all governed by the same mathematical laws. In computer animation, we don't just draw; we simulate the laws of the universe."

— ED Catmull (Founder of Pixar Studios)

Thank You!